DEPLOY_FOR_PRESENTATION

NetworkPeer — Presenting from a Locked-Down Linux Laptop (No Code on That Machine)

The presentation laptop cannot run Node/Docker, so the demo must be **browser-only** on that machine: it just opens URLs (one normal window for the client, one incognito for the worker). Everything else runs elsewhere.

> **If you CANNOT bring the Mac to the office** (Mac not allowed), skip this tunnel-based guide and use `DEPLOY_CLOUD_LINUX_ONLY.md` — the full cloud deployment (Railway + Vercel) that works from any browser with zero dependency on the Mac.

Repository: `github.com/addy9087/Networkpeer` — branch `main`, commit `2796fef` (pushed, CI running).

TL;DR — what to do right now

1. **Tonight (this presentation): Option A** — Cloudflare quick tunnels from this Mac (~20 min, zero new accounts). The office laptop opens two public URLs in a browser. Done.
2. **Tomorrow (real, always-on): Option B** — deploy from GitHub to Railway/Render + Vercel (~1–2 h; needs accounts, Twilio for login OTP, AWS S3, Stripe test keys).
3. Never plan to run code on the office laptop. At most, email/Teams the two guides (`DEPLOY_FOR_PRESENTATION.md` ,

`EXECUTIVE_PRESENTATION_AND_DEPLOYMENT_GUIDE.md`) as a presenter reference, not as a runtime.

Option A — Live demo via Cloudflare quick tunnels (do this tonight)

The Mac runs the stack; Cloudflare gives both the API and the frontend public HTTPS URLs. The Linux laptop needs nothing but a browser + internet.

A1. Install cloudflared on the Mac (one time)

```
brew install cloudflared
```

A2. Start the stack (two terminals on the Mac)

```
# Terminal 1 — API (keep open)
cd /Users/adityasharma/Desktop/NETWORKPEER/NetworkPeer-main
npm run build && node dist/index.js

# Terminal 2 — frontend dev server (keep open)
cd /Users/adityasharma/Desktop/NETWORKPEER/NetworkPeer-platform-
        main
npm run dev
```

A3. Open the tunnels (two more terminals on the Mac)

```
# Terminal 3 — API tunnel; copy the printed
        https://xxx.trycloudflare.com URL → call it API_URL
cloudflared tunnel --url http://localhost:3000

# Terminal 4 — frontend tunnel; copy the printed URL → call it
        FE_URL
cloudflared tunnel --url http://localhost:8080
```

A4. Point the frontend at the API tunnel, and the API at the frontend tunnel

```
# Terminal 2 again — restart the frontend with the API tunnel
        baked in:
cd /Users/adityasharma/Desktop/NETWORKPEER/NetworkPeer-platform-
        main
VITE_API_BASE_URL="API_URL/api/v1" VITE_API_PREFIX=/api/v1 npm
        run dev
# (replace API_URL with the actual tunnel URL from Terminal 3)
```

```
# Terminal 1 again — add the frontend origin to CORS and restart
        the API:
#   edit NetworkPeer-main/.env →
        CORS_ORIGINS=http://localhost:8080,FE_URL   (replace
        FE_URL)
#   then restart the API command from A2.
```

A5. Verify before the meeting (from the Mac or office laptop)

```
curl -fsS "API_URL/api/v1/live"          # → {"success":true,...}
curl -fsS "API_URL/api/v1/health"        # → database + postgis
        true
```

Open `FE_URL` in a normal and an incognito window → sign up client/worker (OTP is echoed on-screen in dev mode) → provision admin + verify worker (Step A6) → run the demo script (`EXECUTIVE_PRESENTATION_AND_DEPLOYMENT_GUIDE.md` §4). Settle funding with:

```
cd /Users/adityasharma/Desktop/NETWORKPEER/NetworkPeer-main
node scripts/simulate-payment-webhook.mjs <operationId>
        <providerReference>
```

A6. One-time setup SQL (run on the Mac)

```
cd /Users/adityasharma/Desktop/NETWORKPEER/NetworkPeer-main
docker exec networkpeer-postgis psql -U postgres -d networkpeer -
        f scripts/provision-admin.sql
```

Then verify the worker (use the phone numbers you signed up with):

```
docker exec networkpeer-postgis psql -U postgres -d networkpeer \
  --set=admin_phone="+1XXXXXXXXXX" \
  --set=worker_phone="+1XXXXXXXXXX" <<'SQL'
SELECT public.admin_set_worker_verification(
  (SELECT id FROM public.users WHERE phone_number =
        :'admin_phone'),
  (SELECT id FROM public.users WHERE phone_number =
        :'worker_phone'),
  'VERIFIED', TRUE, 'Verified for live demonstration');
SQL
```

A7. Caveats (tell yourself before the meeting)

- The tunnel URLs are **random each run** — do not restart the tunnels before/during the demo.
- The Mac must stay awake, online, and not sleep.

- Evidence upload (demo Step C) needs S3; without it, skip that step or use a real bucket (instructions in `DEPLOY_TONIGHT.md`).
- Corporate network on the office laptop must allow HTTPS outbound (it will).

Option B — Always-on deployment from GitHub (finish tomorrow)

Requirements: GitHub repo connected to the providers, plus **Twilio** (cloud login needs real SMS — `OTP_ECHO_IN_RESPONSE=true` and `SMS_PROVIDER=console` are **rejected in production**), AWS S3, and Stripe test keys. Without Twilio, a production cloud login cannot work; the tunnel option does not have this problem.

B1. Backend on Railway (or Render/Fly)

1. New project → **Deploy from GitHub repo** `addy9087/Networkpeer`.
2. Create two services from root directory `NetworkPeer-main` (build `npm ci && npm run build`):
   - **API**: start command `node dist/index.js`, env `BACKGROUND_QUEUES_ENABLED=false`.
   - **Worker**: start command `node dist/background-worker.js`, env `BACKGROUND_QUEUES_ENABLED=true`.
3. Add managed **PostgreSQL** (name `networkpeer`, owner can `CREATE EXTENSION postgis`) and **Redis** (`rediss://` TLS URL).
4. **Before the API gets traffic**, run migrations + provisioning once from the Mac using the provider's migration-owner URL:

```
cd /Users/adityasharma/Desktop/NETWORKPEER/NetworkPeer-main
npm ci
export DATABASE_URL="$DATABASE_MIGRATION_URL" && npm run
        migrate
psql "$DATABASE_MIGRATION_URL" -v ON_ERROR_STOP=1 -f
        scripts/provision-app-role.sql
```

5. Build the four runtime DB URLs (`networkpeer_app`, `networkpeer_admin_api`, `networkpeer_media_verifier`, `networkpeer_financial_api`) from the provider's host + the passwords you generated; all with `sslmode=require`.

6. Set the full env table from `EXECUTIVE_PRESENTATION_AND_DEPLOYMENT_GUIDE.md` §2.5 (JWT secrets via `openssl rand -hex 48`, Twilio, AWS S3, Stripe test keys, `STRIPE_WEBHOOK_SECRET` from `stripe listen`, `CORS_ORIGINS` = exact Vercel origin).

7. Verify `GET https://<api>/api/v1/live` → 200 before deploying the frontend.

B2. Frontend on Vercel

1. New project from the same repo, root directory `NetworkPeer-platform-main`.
2. Node.js 24; build command `npm run build`; env `NITRO_PRESET=vercel`, `VITE_API_BASE_URL=https://<api>/api/v1`, `VITE_API_PREFIX=/api/v1`.
3. Copy the exact HTTPS origin → set backend `CORS_ORIGINS` → redeploy API + worker.

B3. Payments and evidence

- Funding in production requires **Stripe test mode**: run `stripe listen --forward-to https://<api>/api/v1/webhooks/payments` on the Mac during the demo and confirm the payment intent with `stripe payment_intents confirm <id> --payment-method pm_card_visa` (guide §4.1/§4.2). The signed-webhook simulator is dev-only.
- Evidence requires the real S3 bucket setup (`DEPLOY_TONIGHT.md` "S3 for the evidence step" section, with the production frontend origin in the bucket CORS).

B4. What GitHub already gives you

- Pushed commit `2796fef` → GitHub Actions runs lint, typecheck, build, unit + E2E tests, and a Docker build on every push to `main` (watch it pass at `github.com/addy9087/Networkpeer/actions`).
- The Dockerfile and `docker-compose.prod.yml` are in-repo for self-hosting later.

Option C — File transfer fallback (only if both options fail)

The office laptop can't run this stack (SSR frontend + API + PostGIS + Redis). Sending the built files only works if Node.js exists on that machine, which is unlikely on a locked-down corporate laptop:

```
# Mac: build the frontend server bundle
cd NetworkPeer-platform-main && npm run build
# copy .output to the office laptop, then there (needs Node):
npx vite preview --host 0.0.0.0
```

If Node is blocked, email/Teams the two markdown guides and present a recorded/screenshot demo instead. Do not rely on this path for the presentation.

Demo day checklist

- [ ] Mac on + online, API terminal + frontend terminal + 2 tunnel terminals open
- [ ] `curl -fsS "API_URL/api/v1/live"` returns 200
- [ ] `FE_URL` opens on the office laptop; both roles sign up (OTP echoed)
- [ ] Admin provisioned, worker verified (A6)
- [ ] Stripe/S3 steps decided: skip evidence, or bucket ready
- [ ] Demo script (`EXECUTIVE_PRESENTATION_AND_DEPLOYMENT_GUIDE.md` §4) printed/visible
- [ ] Funding settlement command ready (webhook simulator for Option A, Stripe CLI for Option B)